Document number: P3338R0
Audience: EWG

Ville Voutilainen
2024-06-23

Observe and ignore semantics in constant evaluation

Abstract

This paper explains why the observe and ignore semantics are useful in constant evaluation, i.e. for contract assertions that are evaluated at translation time.

The long and short of it

So, we start out with a function:

```
int f(int x)
{
    if (x < 0) // defensive check
      return -1;
    return x * 2; // a totally concocted example definition, in
reality much more complex
}
```

Okay, now we add a contract

```
int f(int x) pre(x >= 0)
{
    if (x < 0) // defensive check
      return -1;
    return x * 2; // a totally concocted example definition, in
reality much more complex
}
```

Not all callers have necessarily accommodated the contract, and the defensive check is still there. So callers that call with negative numbers work fine. To find them, you evaluate the pre with the 'observe' semantic. Incorrect callers are incorrect, but an incorrect call is perfectly recoverable.

Make it constexpr,

```
constexpr int f(int x) pre(x >= 0)
{
    if (x < 0) // defensive check
      return -1;
    return x * 2; // a totally concocted example definition, in
reality much more complex
}
```

and nothing changes. Incorrect calls are recoverable. Evaluate the pre with the 'observe' semantic, and your program compiles fine. The defensive check protects the innocent.

And then, if you don't want any diagnostics, you compile with the 'ignore' semantic, and you don't even get a warning. Quite like you wouldn't get any violation handler call with a run-time check.

Yes, to use such an approach effectively, you eventually want a label that can mark a contract as "ignore or observe, but not enforce". But it can be used without labels already.